

# 14. Pipeline Orchestration Pattern

Phase: J

Code: `bilingual_etl/scripts/main_etl.py`

## What it is

An **orchestrator** is a piece of code whose entire job is to call *other, already-built* pieces of code in the right order, hand data between them correctly, and decide what to do when one of those pieces fails. It does not invent new logic. It does not know how to translate text, how to talk to an embedding API, or how to build a database transaction — those things already exist. What it knows is the *sequence*: do this, then this, then this, and if step 4 blows up, don't crash the whole thing — log it and move on.

Every individual capability that Phase J's orchestrator calls was already built and independently tested in an earlier phase of this project:

| Capability                                               | Built in |
|----------------------------------------------------------|----------|
| Extracting raw records from source JSON                  | Phase B  |
| Cleaning HTML out of descriptions                        | Phase B  |
| Deciding whether a record needs reprocessing (hash gate) | Phase H  |
| Translating Hungarian text to English                    | Phase C  |
| Calling an LLM to generate enrichment fields             | Phase C  |
| Masking/unmasking protected terms                        | Phase D  |
| Lemmatizing text for BM25 search                         | Phase E  |
| Building narrative prose from structured data            | Phase G  |
| Chunking long company narratives                         | Phase G  |
| Batching + retrying embedding calls                      | Phase H  |
| Writing to the database as an all-or-nothing atomic swap | Phase I  |

None of that changed in Phase J. `main_etl.py` is 327 lines, and almost none of those lines contain new business logic — they are almost entirely *calls into* the functions built in Phases B–I, in a specific order, with the outputs of one step becoming the inputs of the next. That's the whole pattern: **coordination, not invention.**

## Real-life analogy

Think of a restaurant kitchen on a busy night. The chef who calls out orders — "table 7 needs the salmon, fire the grill, then plate it, then pass it to the runner" — doesn't personally grill the salmon, doesn't personally know the exact plating technique, and doesn't personally deliver the food to the table. Each of those jobs belongs to a specialist who already knows how to do their one thing well: the grill cook, the plating station, the runner.

The calling chef's entire skill is sequencing and recovery: knowing that the sauce must reduce *before* the fish goes on the plate, not after — and knowing that if the grill cook burns one order, you don't shut down the whole kitchen for the night. You log it, remake that one plate or apologize to that one table, and every other order still goes out correctly.

`main_etl.py` is that calling chef. `translate()`, `enrich()`, `embed_texts()`, `upsert_category_vectors()` are the specialists.

## Why we used it here

The bilingual ETL pipeline has roughly ten distinct stages, each with its own retry logic, its own external API, its own data shape. Without a single place that owns the *order* of those stages, two things go wrong:

1. **Ordering bugs become invisible.** If translation code and narrative-building code are both free to call each other in ad-hoc ways scattered across the codebase, it becomes very easy to accidentally lemmatize text *before* masking protected terms (forbidden — see the R8 masking-order rule), or to embed a narrative before it has actually been translated.

2. **One bad record takes down the whole run.** A multi-hour batch job processing thousands of companies cannot afford to lose all completed work because record #2,341 threw an exception.

A single orchestrator function makes the order explicit and auditable in one place, and gives us exactly one spot to put failure-isolation logic (see below) so every stage benefits from it, instead of reinventing try/except in ten different files.

## Code walkthrough

### *The top-level sequence*

`run()` at the bottom of `main_etl.py` is the entire pipeline, readable top to bottom:

```
def run(force: bool = False, limit: int | None = None) -> None:
    conn = get_connection()
    masker = ProtectedTermsMasker()

    try:
        categories = extract_categories()
        companies = extract_companies()

        cat_processed, cat_skipped, cat_failed = process_categories(
            conn, categories, masker, force=force, limit=limit
        )

        category_names_lookup = build_category_names_lookup(conn)

        comp_processed, comp_skipped, comp_failed = process_companies(
            conn, companies, masker, category_names_lookup, force=force, limit=limit
        )
        ...
    finally:
        conn.close()
```

Notice what's *not* here: no translation code, no HTTP calls, no SQL beyond opening/closing a connection. It reads as a table of contents for the whole pipeline.

### ***The per-record pipeline order, and why***

Inside `_process_one_category()` and `_process_one_company()`, the stages run in this order:

```
extract -&gt; clean HTML -&gt; hash-gate check -&gt; translate -&gt; AI-enrich -&gt;
build narrative -&gt; (companies only: append synthetic questions, then chunk) -&gt;
mask/lemmatize for BM25 -&gt; embed -&gt; atomic-swap database write
```

Each step depends on the output of the one before it, which is *why* the order is fixed and not arbitrary:

- **Clean HTML before hashing/translating** — you don't want a hash that changes just because someone added a stray `<br>` tag, and you don't want to translate markup tags as if they were words.
- **Hash-gate before translate/enrich** — translation and enrichment are the expensive, rate-limited, paid-API steps. Checking "has the source data actually changed?" first means an unchanged record short-circuits before any API call happens at all.
- **Translate before build-narrative** — the narrative template inserts the translated fields into sentences; it needs the English text to already exist.
- **Build narrative before mask/lemmatize** — lemmatization needs full sentences to do its job correctly, not fragments.
- **Mask/lemmatize before embed** — this is the R8 rule from Phase D: protected terms (product/brand names that must not be altered) are masked before any NLP transformation touches the text, then unmasked afterward. Get this order wrong and you can lemmatize or mis-embed a protected term.
- **Embed and write to DB last** — the database write is the point of no return; everything upstream of it must have already succeeded.

### ***The one non-obvious ordering decision: categories fully before companies***

`run()` calls `process_categories()` to completion, *then* calls `build_category_names_lookup()`, *then* calls `process_companies()`. Categories and companies are independent datasets — nothing stops them from being processed in parallel or interleaved. So why the strict sequence?

A company's narrative needs to say which categories it belongs to, **by name, in the correct language** — e.g. "This company operates in *Industrial Compressors* and *Logistics Services*." If category-name translation happened inline, once per company, a popular category referenced by hundreds of companies would get translated hundreds of times — wasteful, slow, and a needless multiplication of API calls against a rate-limited provider.

Instead, all categories are translated and written to the database first. Then this function builds a lookup table by reading the names *back out of the database*:

```
def build_category_names_lookup(conn) -&gt; dict[str, dict[str, str]]:
    lookup: dict[str, dict[str, str]] = {}
    with conn.cursor() as cur:
        cur.execute("SELECT category_id, language, metadata-&gt;&gt;'category_name' FROM category_vector")
```

```

        for category_id, language, name in cur.fetchall():
            if name:
                lookup.setdefault(category_id, {})[language] = name
    return lookup

```

The important subtlety: this reads the **current database state**, not just "categories processed in this run." That means it works correctly on the very first run (categories were just written moments ago) *and* on every subsequent rerun where most categories are hash-gate-skipped because nothing changed — their names are still sitting correctly in the database from a previous run, and the lookup finds them there just the same. If the lookup only used categories processed in the current run's loop, a rerun where all categories were skipped would leave every company without any category names at all. Reading from the database, not from an in-memory list, is what makes this correct on every run, not just the first.

### ***Per-record failure isolation***

Both `process_categories()` and `process_companies()` loop over their records with a try/except around each individual record:

```

for category in categories:
    try:
        if _process_one_category(conn, category, masker, force):
            processed += 1
        else:
            skipped += 1
    except Exception as e:
        failed += 1
        logger.error(f"Failed to process category_id={category.get('category_id')}: {e}")

```

If one record's processing raises an exception — after its internal retries have already been exhausted — that exception is caught right here, logged with the specific record's ID, counted as a failure, and the loop moves to the next record. The batch keeps going.

**Without this pattern:** an exception raised on record #400 of a 1,000-record run, with no per-record try/except, propagates straight up and kills `run()`. Every successfully processed record before #400 that hadn't yet reached its own database commit could be lost, and the whole multi-hour job has to start over.

**Why this doesn't violate the "never write a partial single-language row" rule:** the database write (`upsert_category_vectors` / `upsert_company_vectors`) is the *last* line of `_process_one_category/_process_one_company`, after translation and enrichment have already both succeeded for both languages. If translation or enrichment throws, execution never reaches the write step at all — nothing partial is ever written. The record is simply skipped in full, logged as a failure, and left for a future rerun (where the hash gate will correctly see it as still needing processing, since its content hash was never successfully recorded).

## **Two real production bugs found only by running the orchestrator end-to-end**

This is the most important lesson of Phase J. Every individual piece — the translator, the enricher, the embedder, the database writer — had already passed its own isolated unit tests in earlier phases. Code

review of `main_etl.py` itself also looked fine. **Neither of those caught either of the following two bugs.** Both only surfaced when the whole pipeline was actually run end-to-end against real data, hitting real external APIs with their real, sometimes-surprising behavior. That is the entire point of integration testing: unit tests prove each piece works *alone*; only running the full system together proves the pieces work correctly *together*, under real conditions.

### ***Bug 1 — A persistent rate-limit wall***

Partway through a real run, Gemini's free tier returned HTTP 429 "Too Many Requests" — and it kept returning it. This was not the brief, one-off blip that the embedding layer's existing retry logic (Phase H: batch, short backoff) was built to smooth over. It was a sustained block that survived all 3 retry attempts with doubling backoff (20 seconds, then 40 seconds — giving up after roughly 60 seconds of total waiting) and still failed. That's a genuine daily quota ceiling, not a hiccup.

The fix in the moment: get a free API key from a second provider (Groq) and flip the *active* provider in the `admin_config` database table. The ETL immediately started working again — **zero code changes**. This is exactly the scenario the Strategy Pattern / provider abstraction (`llm_provider.py`, Phase C) was designed for, and it worked exactly as designed the first time it was actually needed under real pressure, mid-project.

Two other safety guarantees held perfectly through this failure:

- **Atomic-swap (Phase I):** a category that had already succeeded stayed fully and correctly saved, even though the very next record failed. No half-written state.
- **Per-record isolation (this phase):** the company hitting the rate-limit wall repeatedly was logged clearly as a failure; the loop moved on to the next record instead of crashing the batch.

Because this rate-limit gap was *observed*, not hypothetical, `llm_provider.py`'s `generate_with_retry()` was retrofitted with the same batching+retry-with-backoff pattern the embedding layer already had — but with a longer initial backoff (20 seconds, doubling, `GENERATE_MAX_RETRIES = 3`) than the embedding provider uses, because a per-minute rate-limit window genuinely needs more than a few seconds to clear; a short backoff tuned for transient network blips isn't long enough for a quota wall.

### ***Bug 2 — A cross-language leak***

After switching providers, actually reading the generated output (not just checking that the run completed without errors) revealed that some AI-generated content meant for the **English** version of a company profile — the buyer-facing synthetic questions and the recommended headline — came back written in **Hungarian**.

The root cause: the AI enrichment system prompt only explicitly instructed the model to match the input language for *one* field (SEO keywords). Every other free-text field was left to the model's own judgment. Gemini happened to default to matching the input language correctly; Groq, running a different underlying model, did not. The fix was to rewrite the prompt to explicitly require **every** free-text output field to match the input's language, with no exceptions — and, critically, to update the prompt as *stored in the `app_config` database table*, not just its default value in the Python source. The prompt was seeded into the database back in an earlier phase (R24: prompts are editable via the Admin UI without

redeploying code), and a database value seeded before this fix takes priority over the code's default — fixing only the source file would have left the bug live in production.

A smaller, related bug turned up in the same investigation: some short translated phrases came back wrapped in markdown emphasis markers (literal `**`, e.g. `**compressor**`) — again, one provider did this and the other didn't. This mattered because translated text flows straight into narrative prose that gets both embedded as a vector *and* lemmatized into BM25 tokens; stray `**` characters become noise in both paths. It was fixed two ways at once, in `translator.py`:

```
def _strip_markdown_emphasis(text: str) -> str:
    ...

result = _strip_markdown_emphasis(generate_with_retry(llm, prompt, system_prompt=system_prompt))
```

(a) the prompt explicitly tells the model not to use markdown formatting, and (b) the code defensively strips `**bold**`/`*italic*` markers from the output regardless of what the model actually did. Relying purely on an instruction an LLM might not perfectly follow is fragile; the code-level safety net catches whatever slips through. This is the same "don't just ask nicely, also verify/clean the output" philosophy already used elsewhere in this project — stripping markdown code fences from JSON responses back in the enrichment code from an earlier phase.

## The hash-gate + provider-switch interaction

One more detail worth noting because it's a nice proof that two independently-designed features compose correctly: after the provider was switched mid-project, rerunning the orchestrator over the *same* already-processed records correctly **skipped** them via the hash gate, because the underlying source data hadn't changed — even though the active LLM provider had. The hash gate only cares about source content, and the provider switch only cares about which API answers a translate/enrich call; neither one knows or needs to know about the other. They were built separately, for separate reasons, and it worked the first time they had to operate together for real.

## How to explain this in an interview/to a teammate

"Phase J didn't build anything new — it's the orchestrator that wires together everything from earlier phases: extract, translate, enrich, build narrative, embed, and write to the database, in a fixed order, because each step depends on the previous one's output. The one clever bit is processing all categories before any companies, so company narratives can look up category names from the database instead of re-translating the same name hundreds of times. The real value of this phase, though, was running the whole thing end-to-end for the first time and finding two bugs — a sustained rate-limit wall and a cross-language content leak — that no amount of unit testing or code review had caught, because they only existed in how the pieces behaved *together* against real APIs. That's the core lesson: unit tests prove each piece works alone, integration runs prove the system works."

## Files touched in Phase J

- `bilingual_etl/scripts/main_etl.py` — the orchestrator itself

- `bilingual_etl/providers/llm_provider.py` — retrofitted `generate_with_retry()` with 20s-initial doubling backoff, 3 max attempts
- `bilingual_etl/enrichment/translator.py` — `_strip_markdown_emphasis()` defensive cleanup
- `bilingual_etl/enrichment/ai_enrichment.py` — enrichment prompt fields
- `app_config` database table — the live enrichment system prompt, updated to require language-matching on every free-text field (not just the Python source default)